

Reaction Game Using Two ATmega328P Microcontrollers and I²C Communication

Tudor-Andrei Rad

1. Project Assumptions

The objective of this project is to design and implement a reaction-based embedded game system using two ATmega328P microcontrollers communicating through the I²C protocol in a master–slave configuration.

The main assumptions of the project are as follows:

The system consists of two microcontrollers:

- A master microcontroller responsible for user interaction, menu navigation, LCD control, EEPROM data storage, and overall game coordination.
- A slave microcontroller responsible for real-time game execution, LED control, button handling, timing, and score calculation.

Communication between the two microcontrollers is performed using the I²C (TWI) protocol.

User interaction is achieved through:

- An HD44780-compatible LCD display,
- An analog keypad for menu navigation (UP, DOWN, LEFT, SELECT),
- Three physical buttons used during gameplay,
- Three LEDs used as visual reaction targets.

The game supports multiple users, three difficulty levels, and persistent high-score storage using EEPROM memory.

The system must operate reliably across multiple consecutive games and must not block or crash if the player does not press a button during a game round.

2. System Schematic

The hardware schematic of the system was designed and tested using the SimulIDE simulation environment.

The system includes:

- Two ATmega328P microcontrollers,
- One HD44780 LCD display,
- Three LEDs with current-limiting resistors,
- Three push buttons for gameplay,
- One analog keypad connected to an ADC input,
- Pull-up resistors on the I²C data (SDA) and clock (SCL) lines.

The master and slave microcontrollers are connected via the I²C bus, with the master initiating all communication. The slave responds only when addressed.

The schematic clearly separates user-interface functionality from real-time game logic, improving system modularity and reliability.

3. Discussion of Implementation

3.1 System Architecture

The project uses a distributed architecture in which responsibilities are divided between two microcontrollers.

The master microcontroller manages:

- Menu navigation and user selection,
- Difficulty selection,
- LCD display updates,
- EEPROM storage of usernames and high scores,
- Game start and score retrieval via I²C.

The slave microcontroller manages:

- LED activation and switching,
- Button detection,
- Reaction timing using Timer1 interrupts,
- Score calculation based on user performance,
- Transmission of the final score to the master.

This architecture ensures that time-critical operations are isolated from display updates and EEPROM access, which could otherwise introduce delays.

3.2 I²C Communication

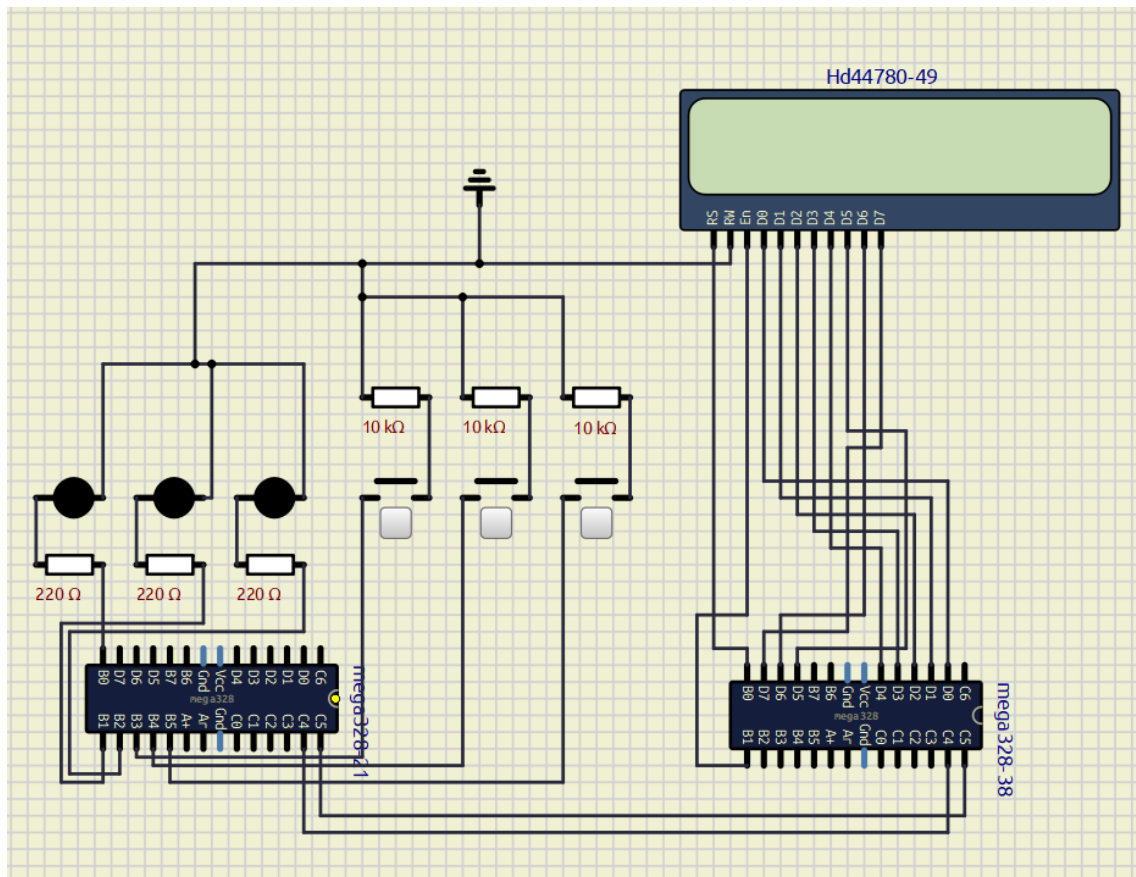
The communication protocol between the master and slave is intentionally simple and robust.

The master sends:

- A command byte indicating game start,
- A byte specifying the selected difficulty level.

After a predefined waiting period, the master requests the final score.

The slave responds with a single byte representing the total score accumulated during the game.



No continuous status polling is used, which simplifies synchronization and reduces I²C bus traffic.

3.3 Game Logic

Each game consists of ten reaction rounds.

For each round:

1. A random LED is activated.
2. A hardware timer is started.
3. The player must press the corresponding button before the timer expires.
4. If the correct button is pressed, points are awarded based on the selected difficulty.
5. If the wrong button is pressed or the timer expires, a penalty or no points are applied.
6. The system automatically advances to the next round.

Hardware timer interrupts are used to ensure precise timing and to prevent blocking delays.

The game always completes exactly ten rounds, even if the player does not press any buttons.

3.4 Difficulty Levels

The system supports three difficulty levels that influence both reaction time and scoring.

Easy mode provides longer reaction times and fewer points per correct press. Medium and hard modes reduce the allowed reaction time while increasing the points awarded.

This allows the same game logic to scale in complexity without code duplication.

3.5 EEPROM Storage and User Management

The master microcontroller supports multiple users.

Username and high scores are stored in EEPROM memory to ensure persistence across power cycles.

The system automatically updates a user's high score when a higher score is achieved.

4. Application Operation

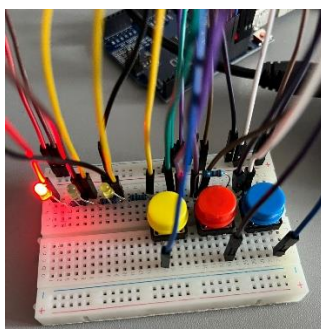
During operation, the user navigates the menu using the analog keypad.



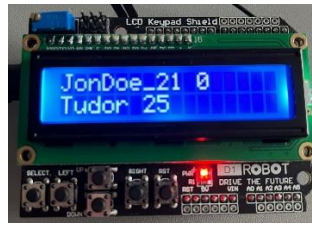
After selecting a user and difficulty level, the game starts with a countdown displayed on the LCD.



The slave executes the game logic independently while the master waits.



Once the game ends, the master retrieves the score and displays it on the LCD, indicating whether a new high score has been achieved.



5. Source Code Repository

The complete source code for both the master and slave microcontrollers is available in my public Git repository.

The repository contains:

- Master source code,
- Slave source code,
- Project documentation,

Repository link:

<https://github.com/tudy0406/Reaction-Time-Tester>

6. Summary

This project demonstrates a complete embedded system using two microcontrollers communicating via I²C in a master–slave configuration.

The system successfully combines real-time processing, user interaction, persistent data storage, and reliable inter-microcontroller communication.

By separating user-interface logic from time-critical game logic, the system achieves both stability and scalability.

The project fulfills all initial requirements and provides a solid foundation for future extensions such as additional game modes, sound feedback, or wireless communication.